

stratum: A System Infrastructure for Massive Agent-Centric ML Workloads

Arnab Phani, Elias Strauss, Sebastian Schelter

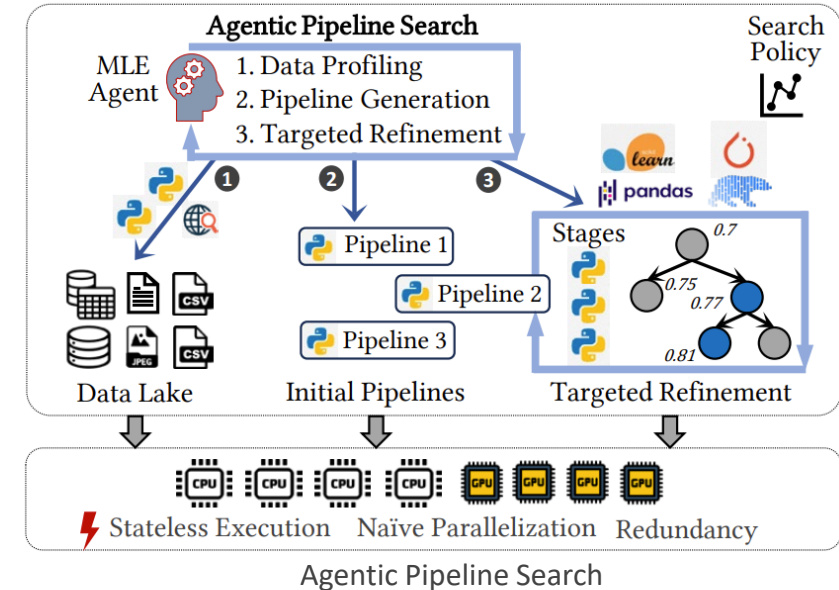
MLE Agents: Agentic Pipeline Search

Example Task Description for an MLE Agent

“Build the best end-to-end ML pipeline to predict customer churn from the datasets in this data lake.”

Simplified Code Generation & Execution Loop

```
for iteration in range(MAX_ITERATIONS):  
    code = agent.generate_or_refine_code()  
    exec(compile(code))  
    agent.evaluate_and_submit()
```



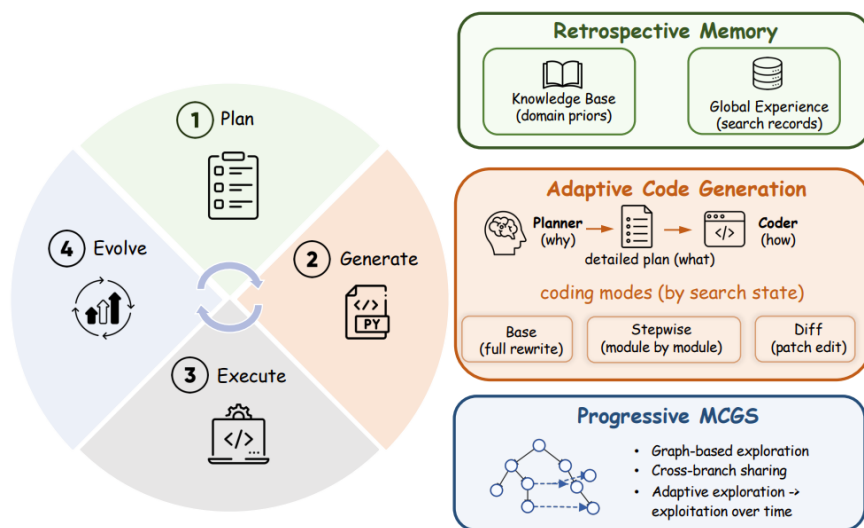
■ LLM-assisted Data Science

- Human-written code -> AutoML -> MLE agents
- Most code in the future will be generated by agents
- ML tasks as code optimization problems
- **Explore many candidate solutions** (in thousands)
- Data profiling, pipeline generation, targeted refinement; repeat

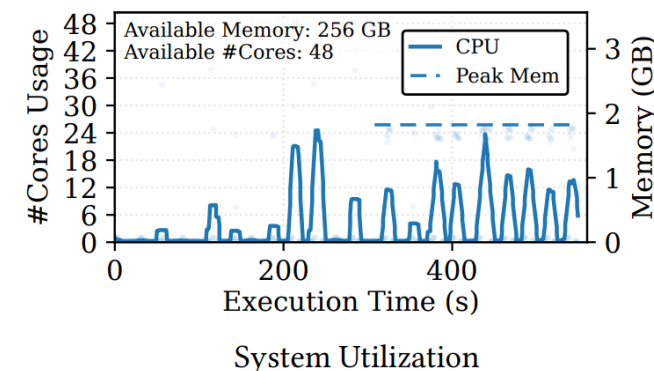
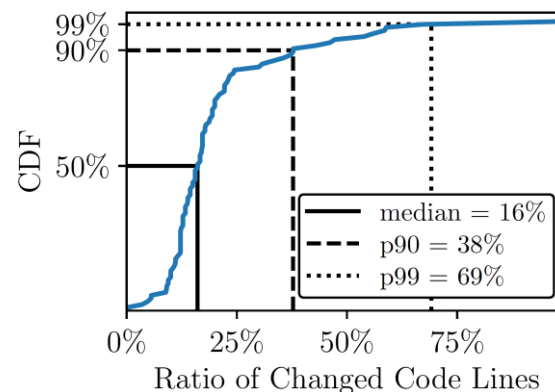


Popular Machine Learning Engineering (MLE) Agents

Challenges of Agentic Pipeline Search



MLEvolve Agent



■ Execution Characteristics of Agentic ML

- Large numbers of **heterogeneous Python code**
- Search strategies combine **exploration and exploitation**
- **System-agnostic** execution
- Redundant computation, inefficient resource utilization, OOM failures...
- Limited or process-based parallelism

MLE agents amplify existing issues as they execute far more pipelines than humans.

Challenges of Agentic Pipeline Search

■ Insufficient ML Libraries

- Built for humans; **usability over efficiency**; eager, sequential
- ML libraries lack optimizing execution layer
- Unmethodical **hand-optimization**
- **Python** is not designed for performance



■ Lack of Efficient Systems for Data Science

- Systems research community made advances
- **DSL-based systems** (SystemDS, OptiML, DAPHNE)
- **Program compilation**, cost-based optimization, multi-backend runtime
- Lack of open code for LLMs to train on
- **Task-specific systems** (Polars, XGBoost, PyTorch) cannot accelerate end-to-end data science pipelines



We still lack an efficient system for end-to-end data science pipelines within Python.

The Case for a New System: Design Principles

- **A System for Agentic Pipeline Search**

- Raising optimization from individual pipeline to pipeline search

- **Extensibility**

- Unrestricted support for ML and data frame libraries, DNN frameworks, TFMs, and specialized libraries
- No operator porting, glue code, decorators, or DSLs

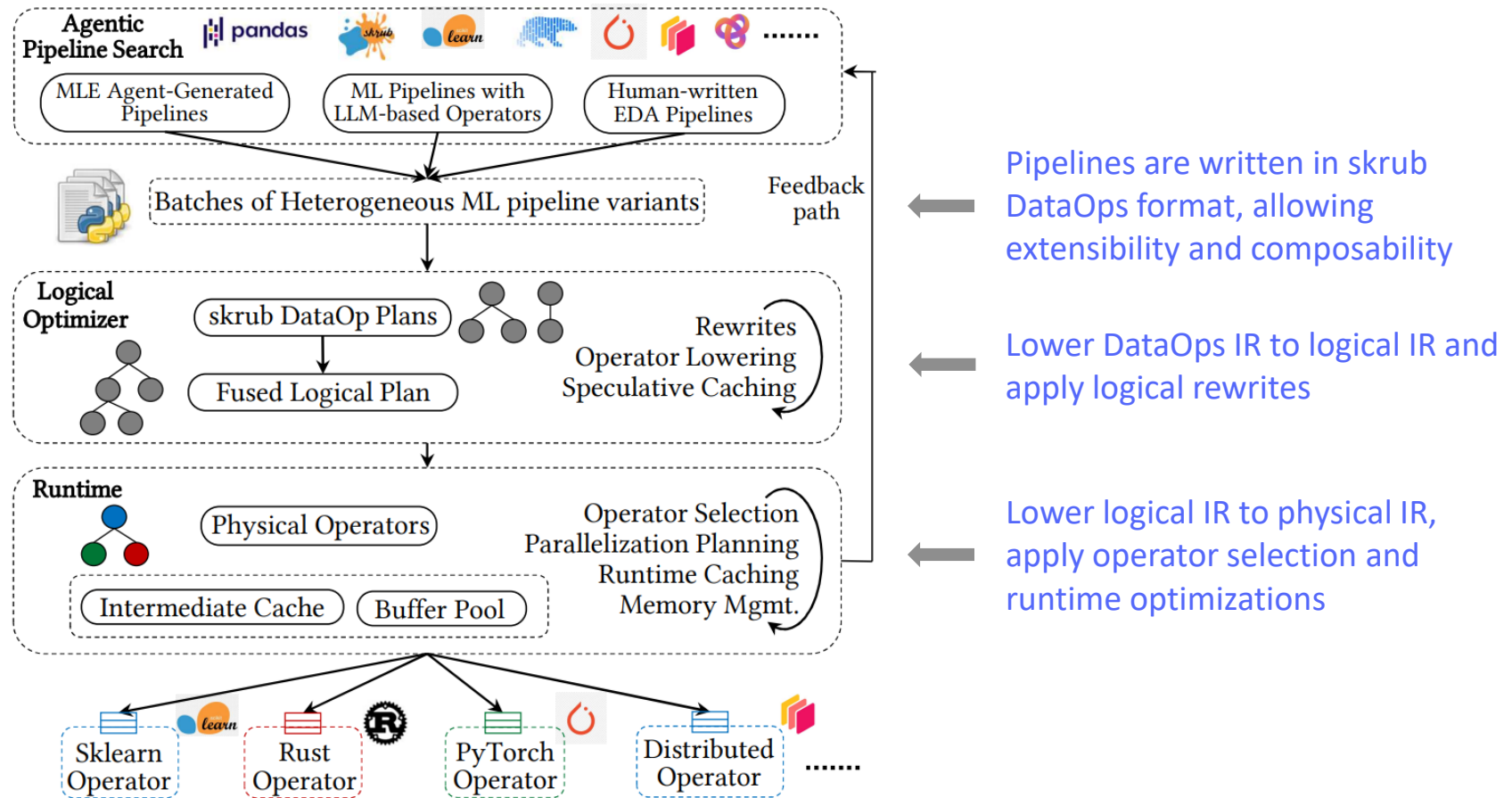
- **Semantic Abstractions**

- Minimal set of logical operators
- Decoupling logical and physical operators
- Lazy evaluation

- **Efficient Runtime**

- Efficient operator kernels; release GIL; parallelism; reuse of intermediates
- Memory management: intermediates, cache, states

A Vision for stratum

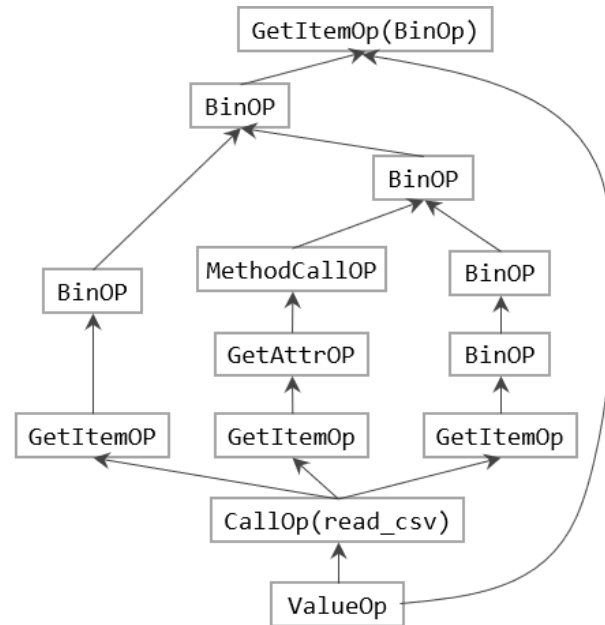


■ stratum

- A system for agentic pipeline search
- Population of evolving heterogeneous ML pipelines



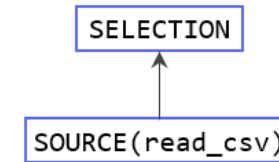
Logical Operators & Rewrites



Skrub
DataOps



Logical
Ops



SELECT

```
src = ...skb.apply_func(pd.read_csv)
pred = (src["age"] >= 25) & (src["age"] < 45) \
      & (src["name"].str.len() > 3) \
      & (src["income"] > 2000)
src_filtered = src[pred]
```

SQL-equivalent

```
SELECT *
FROM train
WHERE age >= 25
      AND age < 45
      AND LENGTH(name) > 3
      AND income > 2000;
```

■ Logical Operators and Rewrites

- Compile (raise) DataOps to Logical operators
- Minimal set of operators (e.g., relational, TRANSFORMER, ESTIMATOR)
- Apply **logical rewrites**

Physical Operators & Runtime Optimizations

■ Physical Operators

- Lower each logical operator to one or more physical operators

■ Registry for Implementations

- A register maintaining **multiple implementations** for each operator
- **Extensibility** for new libraries
- Every operator has a default implementation

■ Runtime Optimizations

- Op selection to **minimize compute time** under memory budget
- **Parallelization planning** and thread distribution
- Intermediate **caching**
- Memory management (diverse types, model states)

Physical Plan

```
=== Plan: physical_impl ===

[00] PolarsReadCSV(/tmp/tmphqrel2uy.csv) [df] <- ()
[01] PolarsSelectionOp(mask) [df] <- ([00])
[02] PolarsDropOp [df] <- ([01])
[03] ImplOp(<X>) <- ([02])
[04] PolarsColumnProjectionOp(cols['target']) <- ([01])

CV Loop:
[05] Split(Train/Test) [df] <- ([03], [04])

Fit Phase:
[06] SplitOutput(y) [df] <- ([05])
[07] SplitOutput(X) [df] <- ([05])
[08] PolarsAssignMapOp(assign: ...) [df] <- ([07])
[09] PolarsColumnProjectionOp(cols[['...']] [df] <- ([08])
[10] StratumFusedTableVectorizer [df] <- ([09])
[11] Ridge [df] <- ([10], [06])

Transform / Predict:
[12] SplitOutput(y) [df] <- ([05])
[13] SplitOutput(X) [df] <- ([05])
[14] PolarsAssignMapOp(assign: ...) [df] <- ([13])
[15] PolarsColumnProjectionOp(cols[['...']] [df] <- ([14])
[16] StratumFusedTableVectorizer [df] <- ([15])
[17] Ridge [df] <- ([16], [12])

Total: 5 pre-split | 1 split | 6 fit | 6 predict
```

Rust Backend

■ Problem

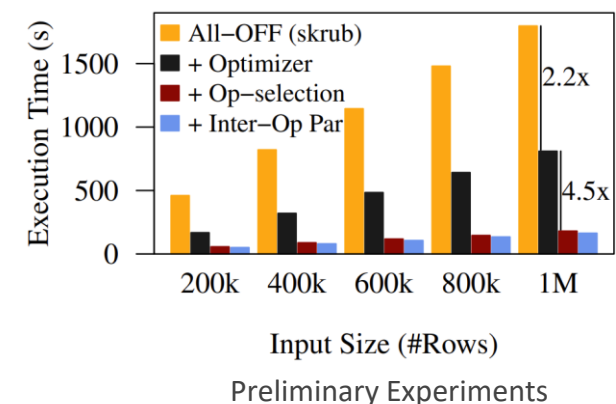
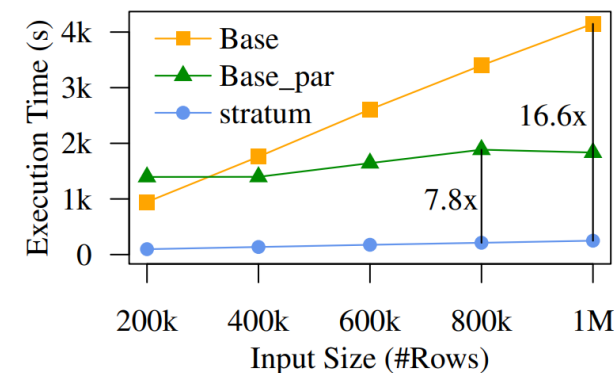
- Scikit-learn and Pandas are slow
- Type conversion, temp allocation, redundant copies, no multithreading
- Usability focused; support a large set of parameters
- Numpy / Cython does not help

■ Rust Backend

- Rust implementations for frequently used operators
- Lightweight PyO3 binding; **release GIL**
- Efficient kernels, **zero-copy** data access, **multi-threaded**
- Support a subset of hyperparameters
- Enables accurate cost estimation, reduced boundary crossings, operator fusion, fine-grained reuse

Table 3: Top learners and transformers in explicit pipelines.

	Top Transformers	Top Learners
SKLEARN	StandardScaler	LogisticRegression
	CountVectorizer	MultinomialNB
	TfidfTransformer	SVC
	PolynomialFeatures	LinearRegression
		RandomForestClassifier



Preliminary Experiments

Key Directions and Challenges

■ Key Directions

- Build stratum
- **Co-optimizing search and execution** (abstract pipeline spec., overlapping generation and exec)
- **Multi-tenant** hosted server (dynamic scheduling, global resource management, ...)
- **Tailored inference engine** for ML code generation (e.g., shared KV caches); local inference
- Unified memory management for agentic memory, KV cache, stratum memory
- Tailor to **health care** / medical use cases

■ Open Challenges

- Heterogeneous operator granularities (fine-grained relational to large model trainings)
- Execution and memory cost estimates
- Black-box UDFs of custom libraries
- Multimodal inputs; DNN workloads; multi-GPU training; distributed backends

Stratum: A System for Agentic ML Pipeline Search



The stratum team

Paper



Code



Personal Website

